COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010/Data Structures and Algorithm | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | September 17, 2026 | **DATE SUBMITTED:** | September 19, 2026 |
| **NAME:** | Gresola, Aryanna Keith G | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. 9.1** **Tree ADT**

## 1. PROCEDURE OUTPUT

*ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.*

**TASK 1**

**INPUT - TREE.H**

9P1_SUPPLEMENTARY > C TreeSample.h

```
1   #ifndef TREE_H
2   #define TREE_H
3
4   #include <iostream>
5
6   template <typename T>
7   class Tree{
8
9       private:
10          T key;
11          Tree<T>* firstChild;
12          Tree<T>* nextSibling;
```

```
62      public:
63      //constructor:
64      Tree(T value);
65
66      //insertion:
67      void insertChild(T value);
```

```cpp
//define:
template <typename T>
Tree<T>::Tree(T value){
    key = value;
    firstChild = nullptr;
    nextSibling = nullptr;
}

// inserting a child:
template <typename T>
void Tree<T>::insertChild(T value){
    // creates a new child
    Tree<T>* newNode = new Tree<T>(value);

    // if there's no child:
    if(firstChild == nullptr){
        firstChild = newNode;
    }else{
        Tree<T>* current = firstChild;

        while(current->nextSibling != nullptr){
            current = current->nextSibling;
        }
        // adding the new node at the end
        current->nextSibling = newNode;
    }
}
```

## INPUT - MAIN.CPP

9P1_SUPPLEMENTARY > main.cpp > ...

```cpp
#include <iostream>
#include "Tree.h"

int main(){

    //create root:
    Tree<char>* tree = new Tree<char>('A');

    // children of A:
    tree->insertChild('B');
    tree->insertChild('C');
    tree->insertChild('D');
    tree->insertChild('E');
    tree->insertChild('F');
    tree->insertChild('G');

    // child of D:
    Tree<char>* D = tree->search('D');
    D->insertChild('H');

    // child of E:
    Tree<char>* E = tree->search('E');
    E->insertChild('I');
    E->insertChild('J');

    // children of J:
    Tree<char>* J = tree->search('J');
    J->insertChild('P');
    J->insertChild('Q');

    // children of F:
    Tree<char>* F = tree->search('F');
    F->insertChild('K');
    F->insertChild('L');
    F->insertChild('M');

    // child of G:
    Tree<char>* G = tree->search('G');
    G->insertChild('N');
```

**TASK 2**

| NODE | HEIGHT | DEPTH |
|---|---|---|
| A | 3 | 0 |
| B | 0 | 1 |
| C | 0 | 1 |
| D | 1 | 1 |
| E | 2 | 1 |
| F | 1 | 1 |
| G | 1 | 1 |
| H | 0 | 2 |
| I | 0 | 2 |
| J | 1 | 2 |
| K | 0 | 2 |
| L | 0 | 2 |
| M | 0 | 2 |
| N | 0 | 2 |
| P | 0 | 3 |
| Q | 0 | 3 |

*ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.*

**TASK 3**

**INPUT - TREE.H**

```
72      void preOrder();
73      void inOrder();
74      void postOrder();
```

```
130 // pre-order:
131 template <typename T>
132 void Tree<T>::preOrder(){
133     std::cout << key << " ";
134     Tree<T>* current = firstChild;
135 
136     while(current != nullptr){
137         current->preOrder();
138         current = current->nextSibling;
139     }
140 }
141 
142 // in order:
143 template <typename T>
144 void Tree<T>::inOrder(){
145 
146     if(firstChild != nullptr){
147         firstChild->inOrder();
148     }
149 
150     std::cout << key << " ";
151     Tree<T>* current = nullptr;
152 
153     if(firstChild != nullptr){
154         current = firstChild->nextSibling;
155     }
156 
157     while(current != nullptr){
158         current->inOrder();
159         current = current->nextSibling;
160     }
161 }
```

```cpp

// post order:
template <typename T>
void Tree<T>::postOrder(){

    Tree<T>* current = firstChild;

    while(current != nullptr){
        current->postOrder();
        current = current->nextSibling;
    }
    std::cout << key << " ";
}
```

**INPUT - MAIN.CPP**

```cpp
    // TASK 3.2:
    std::cout << "--- TREE TRAVERSAL (OUTPUT) ---" << std::endl;

    // pre-order:
    std::cout << "Pre-Order Output: ";
    tree->preOrder();
    std::cout << std::endl;

    // in-order:
    std::cout << "In-Order Output: ";
    tree->inOrder();
    std::cout << std::endl;

    // post-order:
    std::cout << "Post-Order: ";
    tree->postOrder();
    std::cout << std::endl;
```

**TASK 3.1**

*Predicted order output by hand:*

| | |
|---|---|
| **PRE-ORDER** | A B C D H E I J P Q F K L M G N |
| **IN-ORDER** | B A C H D I E P J Q F K L M N G |
| **POST-ORDER** | B C H D I P Q J E K L M F N G A |

## TASK 3.2

*Actual console output*

### OUTPUT

```
--- TREE TRAVERSAL (OUTPUT) ---
Pre-Order Output: A B C D H E I J P Q F K L M G N
In-Order Output: B A C H D I E P J Q K F L M N G
Post-Order: B C H D I P Q J E K L M F N G A
```

### OBSERVATION

When I compared my prediction with the result on the console, I observed that knowing the pattern and how the tree is traversed makes it possible to predict the output correctly. The result from the code was the same as my prediction, which shows that the code follows the same traversal process. As long as the tree structure and traversal pattern are understood and the code is implemented properly, we can get the expected result.

## TASK 3.3

### INPUT - TREE.H

```
12      bool findPreOrder(T value){
13          if(key == value){
14              return true;
15          }
16          Tree<T>* current = firstChild;
17          while(current != nullptr){
18              if(current->findPreOrder(value)){
19                  return true;
20              }
21              current = current->nextSibling;
22          }
23          return false;
24      }
```

```
26      bool findInOrder(T value){
27          if(firstChild != nullptr){
28              if(firstChild->findInOrder(value)){
29                  return true;
30              }
31          }
32          if(key == value){
33              return true;
34          }
35          Tree<T>* current = nullptr;
36          if(firstChild != nullptr){
37              current = firstChild->nextSibling;
38          }
39          while(current != nullptr){
40              if(current->findInOrder(value)){
41                  return true;
42              }
43              current = current->nextSibling;
44          }
45          return false;
46      }
47
48      bool findPostOrder(T value){
49          Tree<T>* current = firstChild;
50          while(current != nullptr){
51              if(current->findPostOrder(value)){
52                  return true;
53              }
54              current = current->nextSibling;
55          }
56          if(key == value){
57              return true;
58          }
59          return false;
60      }
```

```
76      // find data:
77      void findData(int choice, T value);
78
79  };
```

```cpp
// find data:
template <typename T>
void Tree<T>::findData(int choice, T value){
    bool found = false;

    // pre-order:
    if(choice == 1){
        found = findPreOrder(value);
    }
    // in order:
    else if(choice == 2){
        found = findInOrder(value);
    }

    // post-order:
    else if(choice == 3){
        found = findPostOrder(value);
    }

    if(found){
        std::cout << value << " was found!" << std::endl;
    }
}

#endif
```

**INPUT - MAIN.CPP**

```cpp
    // TASK 3.3:
    std::cout << std::endl;
    std::cout << "--- FIND DATA ---" << std::endl;

    // choice 1: pre-order
    tree->findData(1, 'G');

    //choice 2: in-order:
    tree->findData(2, 'F');

    //choice 3: post-order:
    tree->findData(3, 'H');
```

**OUTPUT**

When value exist:

```
--- FIND DATA ---
G was found!
F was found!
H was found!
```

When value does not exist:

```
--- FIND DATA ---
```

**TASK 3.4**

**INPUT - MAIN.CPP**

```
73      // TASK 3.4
74      // inserting O as child of G:
75      G->insertChild('O');
76
77      std::cout << std::endl;
78      std::cout << "--- ADDING O AS A CHILD OF G ---" << std::endl;
79
80      tree->findData(1, 'O');
81      // pre-order:
82      std::cout << "Pre-Order Output (after adding O): ";
83      tree->preOrder();
84      std::cout << std::endl;
85
86      // in-order:
87      std::cout << "In-Order Output (after adding O): ";
88      tree->inOrder();
89      std::cout << std::endl;
90
91      // post-order:
92      std::cout << "Post-Order Output (after adding O): ";
93      tree->postOrder();
94      std::cout << std::endl;
```

## OUTPUT

```
--- ADDING O AS A CHILD OF G ---
O was found!
Pre-Order Output (after adding O): A B C D H E I J P Q F K L M G N O
In-Order Output (after adding O): B A C H D I E P J Q K F L M N G O
Post-Order Output (after adding O): B C H D I P Q J E K L M F N O G A
```

## OBSERVATION

After adding O as a child of G, I observed that O becomes a leaf node because it does not have any children. Its child pointers are both NULL since there are no nodes connected to its left or right. After incorporating O into the general tree, O was also included in the traversal results. This shows that adding a new node properly to the tree also updates the output of the traversal.

## OBSERVATION/ANALYSIS

In this part of the activity, I learned how to implement a general tree, which is quite different from a BST because a node can have more than two children. At first, I found it confusing to understand how I would implement this in code, but I was able to figure out how the nodes and their connections work. This helped me understand how a general tree can be represented and traversed in C++.

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**INPUT - BST.H**

BST.h > insert(T)

```h
#ifndef BST_H
#define BST_H

template <typename T>
class BinarySearchTree{
    private:
        T key;
        BinarySearchTree<T>* left;
        BinarySearchTree<T>* right;
    public:
        //constructor:
        BinarySearchTree(T value);

        // insertion:
        void insert(T value);

        //traversal:
        void preOrder();
        void inOrder();
        void postOrder();

};

// define:
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;
}
```

```
31
32  template <typename T>
33  void BinarySearchTree<T>::insert(T value){
34      // go to the left: value < key
35      if(value < key){
36          if(left == nullptr){
37              left = new BinarySearchTree<T>(value);
38          }else{
39              left->insert(value);
40          }
41      }
42      else if(value > key){
43          if(right == nullptr){
44              right = new BinarySearchTree<T>(value);
45          }else{
46              right->insert(value);
47          }
48      }
49      // go to the right: value > key
50  }
51
52  template <typename T>
53  void BinarySearchTree<T>::preOrder(){
54      std::cout << key << " ";
55
56      if(left != nullptr){
57          left->preOrder();
58      }
59      if(right != nullptr){
60          right->preOrder();
61      }
62  }
63
```

```
64  template <typename T>
65  void BinarySearchTree<T>::inOrder(){
66      if(left != nullptr){
67          left->inOrder();
68      }
69      std::cout << key << " ";
70
71      if(right != nullptr){
72          right->inOrder();
73      }
74  }
75
76  template <typename T>
77  void BinarySearchTree<T>::postOrder(){
78      if(left != nullptr){
79          left->postOrder();
80      }
81
82      if(right != nullptr){
83          right->postOrder();
84      }
85      std::cout << key << " ";
86  }
87
88  #endif
89
90
```

## INPUT - MAIN.CPP

main.cpp > ...

```cpp
#include <iostream>
#include "BST.h"

int main(){
    BinarySearchTree<int>* tree = new BinarySearchTree(2); // pivot value

    tree->insert(3);
    tree->insert(9);
    tree->insert(18);
    tree->insert(0);
    tree->insert(1);
    tree->insert(4);
    tree->insert(5);

    std::cout << "--- BINARY SEARCH TREE ---" << std::endl;

    // PRE-ORDER OUTPUT:
    std::cout << "Pre-Order Output: ";
    tree->preOrder();
    std::cout << std::endl;

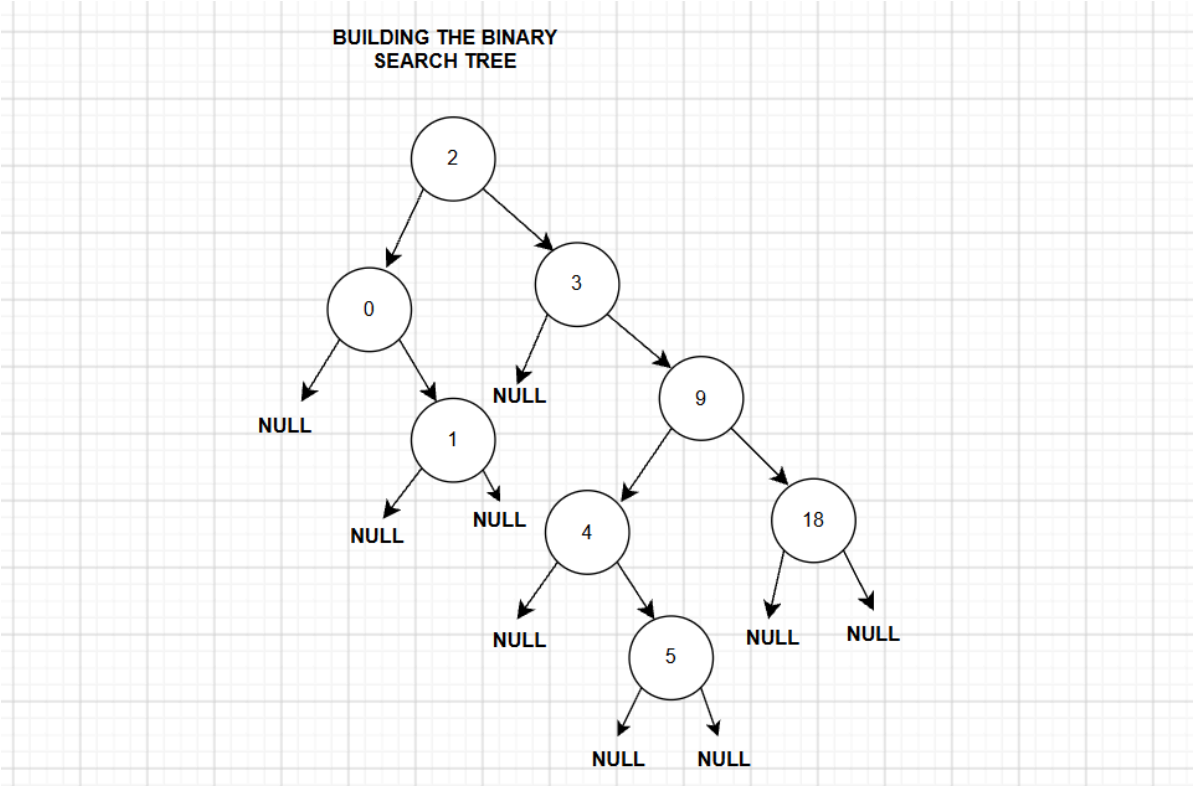
    // IN-ORDER OUTPUT:
    std::cout << "In-Order Output: ";
    tree->inOrder();
    std::cout << std::endl;

    // POST-ORDER OUTPUT:
    std::cout << "Post-Order Output: ";
    tree->postOrder();
    std::cout << std::endl;

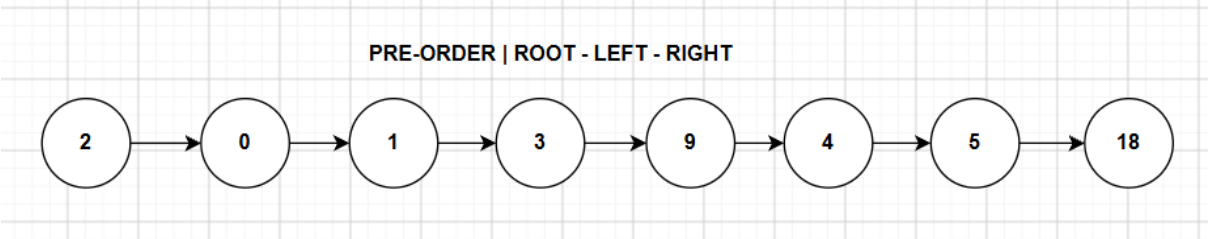
    return 0;
}
```

## DIAGRAMS

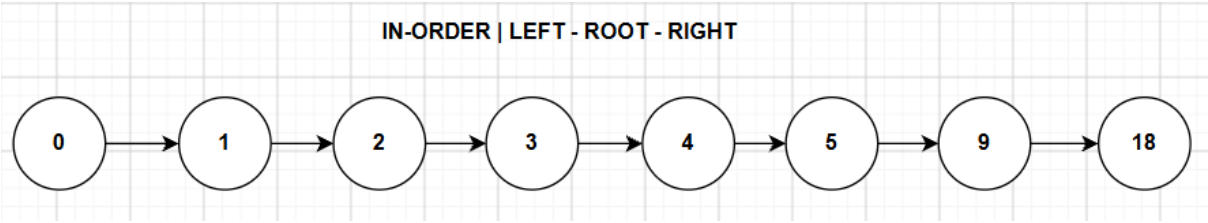
*DEFAULT BINARY TREE DIAGRAM*



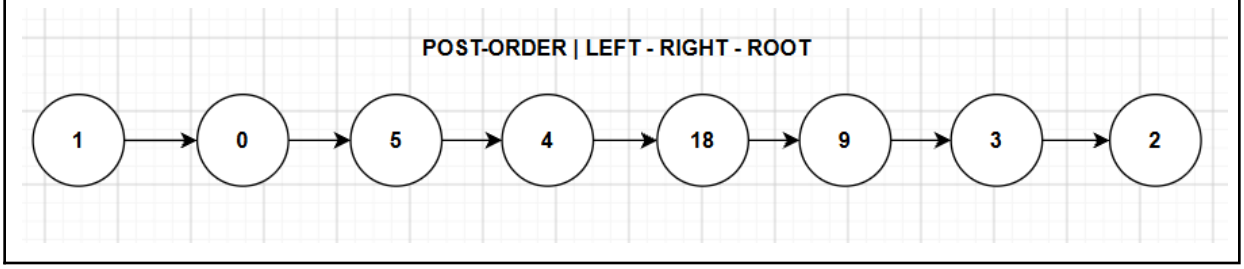

*PRE-ORDER DIAGRAM*




*IN-ORDER DIAGRAM*



*POST-ORDER DIAGRAM*



| FUNCTION | OUTPUT |
|---|---|
| 1. PRE-ORDER: | Pre-Order Output: 2 0 1 3 9 4 5 18 |

```
54  template <typename T>
55  void BinarySearchTree<T>::preOrder(){
56      std::cout << key << " ";
57
58      if(left != nullptr){
59          left->preOrder();
60      }
61      if(right != nullptr){
62          right->preOrder();
63      }
64  }
```

**OBSERVATION**

The pre-order traversal was executed using the preOrder() function in the program. The result is not different from my diagram because it follows the Root - Left - Right strategy. The output matched the result that appeared in the console from the code I created.

| | |
|---|---|
| 2. IN-ORDER: | In-Order Output: 0 1 2 3 4 5 9 18 |

```
66  template <typename T>
67  void BinarySearchTree<T>::inOrder(){
68      if(left != nullptr){
69          left->inOrder();
70      }
71      std::cout << key << " ";
72
73      if(right != nullptr){
74          right->inOrder();
75      }
76  }
```

**OBSERVATION**

The in-order traversal was executed using the inOrder() function in the program. The result is not different from my diagram because it follows the Left - Root - Right strategy. The output matched the result that appeared in the console from the code I created.

3. POST-ORDER:

```
78  template <typename T>
79  void BinarySearchTree<T>::postOrder(){
80      if(left != nullptr){
81          left->postOrder();
82      }
83
84      if(right != nullptr){
85          right->postOrder();
86      }
87      std::cout << key << " ";
88  }
```

```
Post-Order Output: 1 0 5 4 18 9 3 2
```

## OBSERVATION

The post-order traversal was executed using the postOrder() function in the program. The result is not different from my diagram because it follows the Left - Right - Root strategy. The output matched the result that appeared in the console from the code I created.

## OBSERVATION/ANALYSIS

In the supplementary activity, I learned more about how a Binary Search Tree can be implemented using C++ and how values are arranged based on the BST rule. I was able to see how each value is placed on the left or right depending on its relationship with the parent node. The traversal methods also became easier to understand because I could compare the order from my diagram with the actual output from the program. I noticed that each traversal produces a different sequence even though they use the same tree, because they follow different strategies. This activity helped me understand that knowing the concept first makes it easier to check whether the code is producing the expected result. Although I still find implementing a BST in code challenging, doing the supplementary activity helped me understand the connection between the tree diagram, the code, and the output.

COMPUTER ENGINEERING

QUEZON CITY

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** Grammarly and ChatGPT

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** Grammar correction, code assistance, and explanation of concepts.

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** I used Grammarly for grammar improvements and ChatGPT for concept explanations and code assistance. These tools did not replace my original thinking, analysis, authorship, or completion of the required work.

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.